Infrastructure Orchestration & Automation

Bloc 3 — Implémenter et automatiser le déploiement des solutions de données

TP guidé — Jour 3

Provisionner et automatiser

Terraform — GitLab CI/CD

Enseignant	Dr. Abdelhak TOUITI
Durée	3h30 (après-midi)
Modalité	Individuel ou binôme
Compétences visées	C8.2, C7.1
Outils principaux	Terraform, GitLab CI/CD

Fait suite au cours du matin : principes de l'Infrastructure as Code.

Présentation du TP

Ce TP enchaîne les deux outils vus ce matin, dans leur ordre naturel d'utilisation : **Terraform** provisionne plusieurs ressources liées entre elles, puis un pipeline **GitLab CI/CD** valide automatiquement votre code à chaque modification. Aucun compte cloud n'est nécessaire : tout tourne en local, contre votre Docker Desktop, via le provider Terraform dédié à Docker.

Objectifs pédagogiques

- Écrire un projet Terraform à plusieurs ressources et comprendre le cycle `init` / `plan` / `apply`.
- Observer le *state* Terraform et l'idempotence du `plan`.
- Lire le graphe de dépendances que Terraform construit automatiquement.
- Mettre en place un pipeline GitLab CI à plusieurs étapes, utilisant une variable CI/CD.

Prérequis

- Docker Desktop installé et démarré (Jour 2).
- Git installé et un compte gitlab.com (gratuit) pour la partie CI/CD.
- Terraform : pas besoin de l'avoir déjà installé, l'Étape 1 vous guide.
- Un dépôt de travail `tp-iac/` pour l'ensemble du TP.

Livrable attendu

- Le dépôt Git (poussé sur GitLab) contenant : `main.tf`, `variables.tf`, `.gitlab-ci.yml`.
- Une capture d'écran de `terraform apply` réussi.
- Un extrait de la sortie de `terraform graph` avec les relations entre vos trois ressources identifiées.
- Une capture d'écran du pipeline GitLab CI passé au vert, avec ses deux jobs.
- Les réponses aux questions de compréhension (section 8).

Les mots à connaître

Terraform et GitLab CI/CD ont chacun leur vocabulaire. Revenez à ce tableau dès qu'un terme vous échappe.

Fichier `.tf`	Fichier texte qui décrit l'infrastructure voulue, dans le langage HCL de Terraform.
Resource	Un élément d'infrastructure décrit dans le code : ici une image, un réseau, un conteneur Docker.
Provider	Plugin qui apprend à Terraform à parler à un fournisseur précis (ici Docker). Sans provider, Terraform ne sait rien créer.
State	Fichier `terraform.tfstate` : la mémoire de Terraform, qui retient ce qu'il a déjà créé.
Plan / apply	`plan` montre ce qui serait fait, sans rien modifier ; `apply` le fait réellement.
Pipeline	Suite d'actions automatiques que GitLab lance à chaque `git push`.
Stage / job	Un stage est une étape du pipeline ; un job est une tâche à l'intérieur d'un stage.
Runner	Machine, gérée par GitLab, qui exécute vos jobs dans un conteneur Docker.
Variable CI/CD	Valeur enregistrée dans GitLab (hors du code) et transmise aux jobs.

1 Installer Terraform

Étape 1

> **À quoi ça sert ?**
> Terraform est un programme en ligne de commande, un simple exécutable. `winget` est le gestionnaire de paquets de Windows : la première commande télécharge et installe Terraform, la seconde vérifie que l'installation a réussi en affichant sa version.

```
winget install --id Hashicorp.Terraform
terraform version
```

> **Astuce**
> Si la commande n'est pas reconnue après l'installation, fermez et rouvrez votre terminal PowerShell — le PATH n'est relu qu'au démarrage d'un nouveau terminal.

Vérification
`terraform version` doit afficher un numéro de version, sans erreur.

2 Provisionner une infrastructure à plusieurs ressources

Étape 2

Un projet Terraform est un **dossier** contenant des fichiers `.tf`. Terraform lit tous les fichiers `.tf` du dossier comme un seul ensemble ; les noms `main.tf` et `variables.tf` sont une convention, pas une obligation. Créez un dossier `tp-iac/` et placez-y les deux fichiers suivants.

2.1 — `variables.tf`

> **À quoi ça sert ?**
> Ce fichier déclare un **paramètre** que l'on peut changer sans toucher à la description de l'infrastructure. Ici : le port de votre machine sur lequel le site sera accessible. Vous le modifierez à l'Étape 5.

```
variable "external_port" {
  description = "Port expose sur la machine hote"
  type = number
  default = 8080
}
```

Comment lire ce fichier

- `variable "external_port"` : déclare un paramètre nommé `external_port`. Dans `main.tf`, on l'utilisera sous la forme `var.external_port`.
- `description` : une phrase pour les humains. Terraform ne s'en sert pas.
- `type = number` : Terraform refusera une valeur qui n'est pas un nombre.
- `default = 8080` : la valeur utilisée si personne n'en fournit une autre.

2.2 — `main.tf`

> **À quoi ça sert ?**
> C'est le fichier principal : il **décrit l'infrastructure voulue**. Ici, trois ressources liées entre elles : une image nginx, un réseau Docker, et un conteneur qui utilise les deux et affiche la page d'accueil de nginx. Vous n'écrivez jamais "comment faire" (pas de `docker run`) : vous décrivez le résultat, et Terraform déduit les actions.

```
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = ">= 4.0.0"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
```

```
  name = "nginx:latest"
  keep_locally = true
}

resource "docker_network" "tp3_net" {
  name = "tp3-network"
}

resource "docker_container" "web" {
  name = "tp3-web"
  image = docker_image.nginx.image_id

  networks_advanced {
    name = docker_network.tp3_net.name
  }

  ports {
    internal = 80
    external = var.external_port
  }
}
```

Comment lire ce fichier, bloc par bloc

- `terraform { required_providers }` : liste les plugins à télécharger. Ici `kreuzwerker/docker`, le plugin qui sait piloter Docker, dans une version $\geq$ 4.0.0.
- `provider "docker" {}` : configure ce plugin. Les accolades vides signifient : se connecter automatiquement au Docker Desktop de votre machine.
- `resource "docker_image" "nginx"` : télécharge l'image `nginx:latest` (équivalent de `docker pull`). `keep_locally = true` évite que l'image soit supprimée de votre machine lors du nettoyage final.
- `resource "docker_network" "tp3_net"` : crée un réseau Docker nommé `tp3-network`.
- `resource "docker_container" "web"` : crée le conteneur (équivalent de `docker run`). Trois lignes à bien comprendre :
  - `image = docker_image.nginx.image_id` : le conteneur utilise l'image créée plus haut. Cette référence à une autre ressource est ce qui indique à Terraform de créer l'image *avant* le conteneur.
  - `networks_advanced { …}` : attache le conteneur au réseau `tp3-network`. Même principe de référence.
  - `ports { …}` : publie le port 80 du conteneur (celui de nginx) sur le port `var.external_port` de votre machine, soit 8080 par défaut.

Astuce

Dans `resource "docker_container" "web"`, `web` est un nom *interne* au code Terraform, qui sert à référencer la ressource. Le nom réel du conteneur dans Docker est celui de l'argument `name`, soit `tp3-web`.

2.3 — Initialiser, planifier, appliquer

À quoi ça sert ?

Ces trois commandes forment le cycle de base de Terraform, toujours dans cet ordre.

- `terraform init` : télécharge le plugin Docker dans un dossier caché `.terraform/` et crée `.terraform.lock.hcl`, qui fige la version du plugin. À faire une fois par projet.
- `terraform plan` : calcule et affiche ce que Terraform ferait, **sans rien modifier**. C'est votre filet de sécurité.
- `terraform apply` : refait ce calcul, demande confirmation, puis crée réellement les ressources.

```
cd tp-iac
terraform init
terraform plan
terraform apply
```

Tapez `yes` lorsque Terraform le demande.

Comment lire la sortie de `terraform plan`

Chaque ressource est précédée d'un symbole : `+` sera créée, `~` sera modifiée sur place, `-` sera détruite, `-/+` sera détruite puis recréée. La dernière ligne résume le tout, par exemple `Plan: 3 to add, 0 to change, 0 to destroy`. Ici, vous devez voir 3 ressources à créer.

Astuce

Si `provider "docker" {}` ne trouve pas votre Docker Desktop, précisez explicitement le socket : `provider "docker" { host = "npipe:////./pipe/docker_engine" }`.

Vérification

`Invoke-RestMethod http://localhost:8080` doit renvoyer la page d'accueil par défaut de nginx. `docker ps` doit lister le conteneur `tp3-web`, et `docker network ls` doit lister `tp3-network`.

3 Explorer le state

Étape 3

À quoi ça sert ?

Le **state** est la mémoire de Terraform : c'est grâce à lui qu'il sait ce qui existe déjà et ne recrée pas ce qui est en place. `terraform state list` affiche la liste des ressources qu'il suit ; `terraform show` en affiche le détail (identifiants, ports, adresses IP…).

```
terraform state list
terraform show
```

Astuce

Un fichier `terraform.tfstate` est apparu dans `tp-iac/`. Ouvrez-le avec un éditeur de texte — vous y retrouverez, en JSON, exactement ce que `terraform show` vient d'afficher.

Vérification

`terraform state list` doit lister trois ressources : `docker_image.nginx`, `docker_network.tp3_net` et `docker_container.web`.

Point de vigilance

Ne committez jamais `terraform.tfstate` tel quel dans un projet réel — il peut contenir des valeurs sensibles en clair. Pour ce TP local, ajoutez-le tout de même à un `.gitignore` par bonne habitude.

À quoi ça sert ? Un fichier `.gitignore` liste ce que Git doit ignorer. Ici : le state, et le dossier `.terraform/` (plugins téléchargés, volumineux et recréés par `terraform init`). En revanche, `.terraform.lock.hcl` **doit** être committé.

```
"*.tfstate*" | Out-File -FilePath .gitignore -Encoding ascii
".terraform/" | Out-File -FilePath .gitignore -Append -Encoding ascii
```

4 Explorer le graphe de dépendances

Étape 4

À quoi ça sert ?

Vous n'avez jamais indiqué à Terraform dans quel ordre créer les ressources. Il l'a déduit des références que vous avez écrites (par exemple `docker_image.nginx.image_id`). `terraform graph` affiche ces dépendances sous forme de texte, pour que vous puissiez les vérifier.

```
terraform graph
```

Astuce

La sortie est au format DOT (le même langage que Graphviz). Repérez les lignes contenant `docker_image.nginx` et `docker_network.tp3_net` : toutes deux doivent apparaître comme des dépendances de `docker_container.web` — exactement le graphe vu en cours ce matin, le même principe qu'un DAG Airflow. Si Graphviz est installé sur votre poste, `terraform graph | dot -Tpng -o graph.png` en produit une image lisible.

Vérification

Dans le texte produit, vous devez repérer une relation de `docker_image.nginx` vers `docker_container.web`, et une autre de `docker_network.tp3_net` vers ce même conteneur. C'est ce graphe qui a dicté l'ordre de création observé à l'Étape 2.

5 Modifier l'infrastructure

Étape 5

5.1 — Rejouer le plan sans rien changer

> **À quoi ça sert ?**
> Vérifier que votre code et la réalité sont d'accord. Comme tout existe déjà, Terraform ne doit rien proposer : c'est l'**idempotence**, la garantie qu'on peut rejouer le code sans risque.

```
terraform plan
```

5.2 — Changer le port exposé

> **À quoi ça sert ?**
> Pour modifier l'infrastructure, on modifie le **code**, jamais Docker à la main. Terraform compare le nouveau code au state et ne touche qu'à ce qui diffère.

Modifiez la valeur par défaut dans `variables.tf` : remplacez `8080` par `8081`, puis :

```
terraform plan
terraform apply
```

Vérification

En 5.1, Terraform doit répondre `No changes` : c'est l'idempotence en action. En 5.2, le plan doit annoncer un conteneur **recréé** (symbole `-/+`), pas simplement modifié — changer un port force le remplacement du conteneur, pas le réseau ni l'image qui restent inchangés. `Invoke-RestMethod http://localhost:8081` doit maintenant répondre.

6 Automatiser avec GitLab CI/CD

Étape 6

6.1 — Créer le projet

> **À quoi ça sert ?**
> GitLab héberge votre dépôt Git et exécute le pipeline à chaque envoi de code. Il faut donc un projet GitLab pour recevoir votre dossier `tp-iac`.

Sur gitlab.com, créez un nouveau projet vide (*New project* → *Create blank project*), par exemple nommé `tp-iac`.

6.2 — Ajouter une variable CI/CD

> **À quoi ça sert ?**
> Une variable CI/CD est une valeur enregistrée **dans GitLab**, hors du code, que GitLab transmet à chaque job sous forme de variable d'environnement. On y met ce qui change selon l'environnement ou qui doit rester secret. Ici, `APP_ENV` sert d'exemple simple : vous la lirez dans le pipeline à l'Étape 7.

Dans l'interface GitLab : *Settings* → *CI/CD* → section *Variables* → *Add variable*. Créez une variable `APP_ENV`, valeur `production` (laissez *Protected* décoché pour ce TP, afin qu'elle reste disponible quelle que soit votre branche, et choisissez la visibilité *Visible* : ne cochez pas *Masked* pour l'instant, c'est l'objet du bonus).

6.3 — Écrire le pipeline

> **À quoi ça sert ?**
> Le fichier `.gitlab-ci.yml` décrit votre pipeline : quelles tâches GitLab doit lancer, dans quel ordre, et dans quel environnement. Son nom est imposé : GitLab le cherche à la racine du dépôt, et dès qu'il le trouve il lance le pipeline à chaque `git push`.

À la racine de `tp-iac/`, créez `.gitlab-ci.yml` :

```
stages:
  - validate
  - info

terraform-validate:
  stage: validate
  image: hashicorp/terraform:latest
  script:
    - terraform init -backend=false
    - terraform fmt -check
    - terraform validate

afficher-environnement:
  stage: info
  image: alpine:latest
  script:
    - echo "Deploiement cible : $APP_ENV"
```

Comment lire ce fichier

- `stages` : la liste ordonnée des étapes du pipeline. `validate` passe avant `info`.
- `terraform-validate` et `afficher-environnement` : les noms de deux **jobs**, choisis librement. Chaque job indique son `stage`.
- `image` : le conteneur Docker dans lequel le job s'exécute. Le runner le télécharge, vous n'installez rien. Ici, `hashicorp/terraform` contient Terraform ; `alpine` est un mini Linux, suffisant pour afficher un message.
- `script` : les commandes lancées dans l'ordre. Si l'une échoue, le job échoue et le pipeline s'arrête.

Les trois commandes du premier job :

- `terraform init -backend=false` : télécharge le plugin Docker, sans configurer de stockage distant du state, inutile pour valider.
- `terraform fmt -check` : vérifie que vos fichiers `.tf` sont mis en forme selon le style officiel. Il ne modifie rien et échoue si la mise en forme est incorrecte.
- `terraform validate` : vérifie que le code est correct (syntaxe, arguments existants, références valides) sans contacter Docker.

Dans le second job, `$APP_ENV` est remplacé par la valeur saisie à l'étape 6.2.

6.4 — Pousser le code

À quoi ça sert ?

Git enregistre l'historique de votre code ; `git push` l'envoie vers GitLab. L'arrivée du fichier `.gitlab-ci.yml` sur GitLab déclenche automatiquement le pipeline.

- `git init -b main` : transforme le dossier en dépôt Git, avec une branche `main`.
- `git add .` : prépare tous les fichiers à enregistrer (sauf ceux du `.gitignore`).
- `git commit` : enregistre un instantané, avec un message qui explique le changement.
- `git remote add origin` : indique l'adresse du dépôt GitLab. Remplacez `<votre-compte>` par votre identifiant GitLab.
- `git push -u origin main` : envoie la branche `main` vers GitLab ; `-u` mémorise ce choix pour les envois suivants.

```
cd tp-iac
git init -b main
git add .
git commit -m "TP Jour 3 : Terraform et pipeline GitLab CI"
git remote add origin https://gitlab.com/<votre-compte>/tp-iac.git
git push -u origin main
```

Astuce

Au premier `git push`, une fenêtre de navigateur peut s'ouvrir pour vous connecter à GitLab. Si on vous demande un mot de passe dans le terminal, saisissez un *token d'accès personnel* (GitLab → *Preferences* → *Access tokens*, droit `write_repository`) : GitLab n'accepte pas le mot de passe de votre compte pour Git. Ces réglages ne concernent que ce dossier, pas vos autres dépôts.

Astuce

Le stage `info` ne démarre que si `validate` a entièrement réussi — c'est l'ordre des `stages` qui l'impose. À l'inverse, si vous ajoutiez un second job au sein du même stage, les deux tourneraient en parallèle.

Point de vigilance

Ce pipeline **valide** votre code (syntaxe, formatage), il ne lance pas de `terraform apply`. Les runners partagés de GitLab tournent dans le cloud de GitLab, sans aucun accès à votre Docker Desktop local — ils ne pourraient de toute façon pas provisionner `tp3-web`. En conditions réelles, un `apply` automatique se déclenche depuis un runner ayant accès aux identifiants cloud nécessaires, généralement après validation manuelle du plan.

7 Observer le pipeline

Étape 7

À quoi ça sert ?

Vous n'avez rien à lancer : GitLab démarre le pipeline tout seul à chaque `git push`. Cette étape consiste à observer son déroulement, job par job, et à lire ses journaux (*logs*) pour comprendre ce qui s'est passé.

Dans l'interface GitLab, ouvrez votre projet puis *Build* → *Pipelines* (le libellé exact peut légèrement varier selon la version de l'interface). Un pipeline doit être apparu automatiquement suite à votre `git push`.

Vérification

Les deux jobs, `terraform-validate` puis `afficher-environnement`, doivent passer au vert, dans cet ordre. Ouvrez les logs du second : vous devez y lire `Deploiement cible : production` — la preuve que la variable définie dans l'interface GitLab (et non dans le code) a bien été injectée dans le job.

Astuce

Si le job `terraform-validate` échoue sur `terraform fmt -check`, c'est que l'indentation de votre fichier `.tf` diffère du style canonique Terraform. Exécutez `terraform fmt` en local, qui corrige automatiquement la mise en forme, puis recommittez.

Astuce

Si le pipeline reste en *pending*, ou si GitLab affiche un message demandant de valider votre compte, c'est que GitLab exige parfois une vérification d'identité avant d'utiliser ses runners partagés. Suivez le lien proposé, ou prévenez l'enseignant.

8 Nettoyage

Étape 8

> **À quoi ça sert ?**
> `terraform destroy` est l'inverse d'`apply` : il supprime toutes les ressources enregistrées dans le state, en respectant l'ordre inverse des dépendances (le conteneur d'abord, puis le réseau). À faire en fin de TP pour libérer le port et les ressources de votre machine. L'image nginx reste sur votre machine grâce à `keep_locally = true`.

```
cd tp-iac
terraform destroy
```

Tapez `yes` pour confirmer.

Vérification
`docker ps -a` ne doit plus lister `tp3-web`. `terraform state list` doit désormais être vide.

9 Questions de compréhension

1. À l'Étape 5, pourquoi changer le port force-t-il Terraform à **recréer** le conteneur plutôt qu'à simplement le modifier, alors que le réseau et l'image, eux, restent inchangés ?

2. À l'Étape 4, qu'est-ce qui, dans le code de `main.tf`, indique à Terraform que `docker_container.web` dépend de `docker_network.tp3_net` ? L'avez-vous déclaré explicitement quelque part ?

3. Pourquoi le pipeline GitLab CI de ce TP exécute-t-il `terraform validate` plutôt que `terraform apply` ?

4. Pourquoi `APP_ENV` est-elle définie dans l'interface GitLab plutôt que directement dans `.gitlab-ci.yml` ? Que se serait-il passé si vous l'aviez écrite en clair dans le fichier à la place ?

Pour aller plus loin (optionnel)

- Paramétrez `main.tf` pour provisionner plusieurs conteneurs à la fois, via `count` ou `for_each`, et observez comment le graphe de dépendances (Étape 4) s'enrichit en conséquence.
- Ajoutez un troisième stage `deploy` dont le job ne se déclenche que sur la branche `main`, grâce à une règle `rules: - if: $CI_COMMIT_BRANCH == "main"`.
- Marquez `APP_ENV` comme *Masked* dans l'interface GitLab, puis observez comment sa valeur est désormais cachée dans les logs du job. GitLab n'accepte de masquer qu'une valeur d'au moins 8 caractères : `production` convient, `dev` non.
- Explorez `terraform plan -out=plan.tfplan` puis `terraform apply plan.tfplan` : appliquer un plan précédemment calculé, plutôt que d'en recalculer un nouveau au moment de l'apply — la pratique recommandée dès qu'un pipeline CI/CD est impliqué.